Chapter 6: Using Distinctiveness and Distance for Comparative Analyses of Historical Data

Many of the techniques used in text mining did not originate in history; rather, they were developed in fields such as the natural sciences, information science, data science, and later adapted for the analysis of historical documents. This kind of cross-disciplinary borrowing can be highly generative, but it also means that the metrics require careful understanding in both how they are applied and how their results are interpreted. When a method is adapted from another discipline, it may not map perfectly onto the problems of history. Analysts must therefore remain attentive to what the metric actually measures, as well as to the kinds of questions it can and cannot answer when applied to any historical corpora.

In this chapter, we examine two concepts used across many scientific fields to identify and compare important features of data: distinctiveness and distance.[1] Both metrics provide a way to compare collections of texts and identify what makes them similar or different. These texts may be organized by author, decade, speaker, debate, political party, or any other category of interest.

Distinctiveness measures how characteristic a word is within a particular context, helping analysts identify terms that occur unusually often in one text or corpus relative to others. Among the most common algorithms used to understand distinctiveness is "tf-idf.".[2] The way tf-idf measurements reveal distinctiveness is by ranking the uniqueness of a word-document pair in respect to other words and other documents in a corpus. The algorithm was originally developed by mathematician Karen Spärk-Jones in 1972, and it has been used routinely in library science as a tool for automatically assigning keywords to articles on the basis of how unique a keyword is to the author's article and how unusual the term is for the field as a whole.[3]

At a high level, the guiding principle of tf-idf is to assign a high weight to terms that are frequent within a given document but relatively infrequent across the entire collection of documents. In our context, a high weight indicates that a term is particularly distinctive to a specific document, meaning it appears often in that document but is rare in the rest of the corpus. Consider an application to the word "cotton" when tf-idf is used to rank the distinctiveness of terms over time. "Term frequency" (TF) measures how often a term appears in a document, reflecting its local importance. "Inverse document frequency" (IDF) measures how rare a term is across the entire corpus, reducing the weight of common words. Multiplying these values yields the tf-idf score, which quantifies a term's distinctiveness in a given document.

There are multiple ways to define term frequency (TF) in a tf-idf equation. A common and straightforward method is to use the raw count of how many times a word appears in a document. Another approach normalizes term frequency by adjusting for document length, dividing the raw count by the total number of words in the document. Normalization is useful when comparing texts of different lengths, such as speeches from two individuals–one who spoke frequently and one who did not–or when analyzing decades with varying numbers of records.

Several variations of tf-idf exist and may be used for analysis. For this chapter, however, we will use just raw counts for our tf-idf calculations for the sake of simplicity.[4]

Inverse document frequency (IDF) measures how common or rare a word is within a corpus. It is calculated by dividing the total number of documents in the corpus by the number of documents that contain the term, then taking the logarithm of that ratio. The primary purpose of IDF is to down-weigh very common words, making infrequent terms more distinctive. While this lesson removes stop words, doing so is not always necessary because IDF naturally reduces the influence of highly frequent terms while emphasizing those that are more unique to specific documents.

Distance, by contrast, quantify differences between texts by comparing their overall patterns of word use. [write more about distance]

[Mention where these metrics have been used perhaps? Jo?].

Distinctiveness and Distance in Practice

To understand how these metrics can be used for the analysis of language, consider a simple example comparing two well-known children's authors. In this example, we will examine Lewis Carroll's *Through the Looking-Glass* and Beatrix Potter's *The Tale of Peter Rabbit*.[5] We chose these two authors for a practical and pedagogical reason: the authors posses clear, stylistic differences. While both authors write for children, their language differs notably. Carroll often uses playful and invented words and abstract descriptions, while Potter tends to use more concrete vocabulary describing animals, nature, and everyday actions. We hypothesize that these differences will make for distinct distributions that we can measure. To test whether our observations about either author are grounded in the books themselves, we can use tf-idf.

The first step is to download the books that will form the corpus for analysis. We will collect our books from Project Gutenberg, a digital library that provides access to books that have been released for public use. We will download the books with R and store them in "plain text"–a format that contains characters but no special encoding for fonts, images, or layouts, making them easy to process. To download the books we can locate their URLs on Project Gutenberg and pass the URLs to the `readLines()` function.

```
# For accessing Hansard
library(hansardr)

# For data processing and text analysis
library(tidyverse)
library(tidytext)

# For "white-box" statistical functions
library(dhmeasures)

# For arranging plots
library(gridExtra)

# For formatting tables
```

```r
library(kableExtra)
library(knitr)

# URLs from Project Gutenberg
looking_glass_url <- "https://www.gutenberg.org/files/12/12-0.txt"
peter_url <- "https://www.gutenberg.org/cache/epub/14838/pg14838.txt"

# Read the text of *Through the Looking-Glass* into R
# - encoding = "UTF-8" correctly reads punctuation and special characters
# - warn = FALSE suppresses warnings that do not affect the code
looking_glass_text <- readLines(looking_glass_url,
                                encoding = "UTF-8", # Read the text using UTF-8 encoding
                                warn = FALSE) # Suppress nonessential warnings

# Do the same for The Tale of Peter Rabbit
peter_text <- readLines(peter_url,
                        encoding = "UTF-8",
                        warn = FALSE)
```

We can now see the books in the Global Environment panel. However, they are not in a tabular format, so we cannot simply click on them and explore their contents the way we could with the `hansardr` data. Yet, when working with the tidyverse, it is common to organize data into a tabular structure because most tidyverse functions are designed to operate on tables. Therefore, we will convert the text into a table before continuing with the analysis.

```r
# Create a table containing the book text
# - tibble() creates a tidyverse-style table
# - writer stores the author's name
# - booktitle stores the book title
# - text stores the book's contents
looking_glass_df <- tibble(writer = "Lewis Carroll", # Store the author's name
                           booktitle = "Through the Looking Glass", # Store the book title
                           text = looking_glass_text) # Store the book text

# Do the same operations for Peter Rabbit
peter_df <- tibble(writer = "Beatrix Potter",
                   booktitle = "Peter Rabbit",
                   text = peter_text)
```

Vectorizing as Preparation for Measurement

If we wanted a comprehensive list of words characteristic of Lewis Carroll but not of Beatrix Potter, we would begin by representing each book as a frequency count of all the words they contain (e.g., "rabbit," "tea," "queen" for *Through the Looking Glass*, and "rabbit," "garden," "jacket" for *The Tale of Peter Rabbit*).

In this example, we combine the two books into a single dataset so that we only need to apply our analysis steps once. Working with one dataset isn't mandatory but it simplifies our work because the same code can be applied across both of our books simultaneously, rather than repeating the same operations for multiple datasets.

When working with large corpora such as the Hansard debates, our process has been different. The Hansard corpus is very large–larger than the standard memory of a laptop–so we often partition the data into smaller subsets, such as decade partitions, to be processed individually. Partitioning the Hansard data is necessary for running the data on many personal computers. However, it also means that the same data processing steps must often be repeated for each dataset.

In this example, the dataset is small enough that we can combine the books into a single dataset using `bind_rows()`, which will later make it easy to apply methods such as tf-idf.

```
# Combine both texts into a single dataframe
combined_books <- bind_rows(looking_glass_df, peter_df)
```

Books downloaded from Project Gutenberg often contain text that is not part of the book itself, such as markers indicating the beginning and end of the text, copyright notices, and section separators. *Through the Looking-Glass*, for example, begins with three stars (***), which mark the start of the Project Gutenberg transcription.

We use a cleaning step to prevent these formatting symbols from entering our analysis. We first use `unnest_tokens()` to tokenize the books into individual words and then apply a regular expression (often shortened to "regex") that keeps only alphabetical words, thus removing symbols, formatting markers, and other non-word characters from the dataset.

```
# Split the books into individual words
# - unnest_tokens() tokenizes the text into one word per row
# - filter() keeps only alphabetic words
tokens <- combined_books %>%
  unnest_tokens(word, text) %>% # Tokenize the text into words
  filter(str_detect(word, "^[a-z]+$")) # Keep only alphabetic words

# Count the words
word_counts <- tokens %>%
  count(writer, word)
```

Our regex can be read like so:

- `^` - The carrot tells the computer to start matching the word at the beginning of the dataframe row. If the token begins with a number or symbol, it will not be kept in the dataset.
- `[]` - The brackets define the set of characters (e.g. the letters a, b, c) that are allowed in the match.
- `a-z` - The alphabet from a through z. Inside brackets, this tells the computer to match any lowercase letter between a and z.
- `+` - A plus sign meaning "match one or more of the preceding characters." This symbol will match any number of alphabetical characters.

- `$` - The dollar sign tells the computer to match to the end of the string.

The order of these data-processing steps matters. The regular expression we just wrote would filter out any token containing punctuation, including words like `rabbit.`. If applied too early, we could unintentionally filter out important words and distort the analysis. However, `unnest_tokens()` has already separated words from punctuation and placed each token in its own row. As a result, punctuation marks appear as separate tokens, allowing us to safely use this regex to remove rows containing only punctuation. If punctuation were still at the ends of words, we would need to modify this regex to delete the punctuation while keeping the word in tact.

At this stage, we can inspect the data and gather preliminary information. Examining the data before applying statistical methods helps determine whether a research question is supported by evidence or whether initial hypotheses are based primarily on anecdotal impressions rather than the texts themselves.

For our example, an analyst might look for words they already know are associated with each author and determine whether those terms appear uniquely or unusually often in that author's texts.

The following code creates a table that allows us to manually inspect how often our selected words appear for each author.

```
# Reshape the table into a wide format
# - pivot_wider() creates one column for each word
# - values_fill = 0 replaces missing values with zeros
author_word_counts <- word_counts %>%
  pivot_wider(names_from = word, values_from = n, values_fill = 0) # Create one column per

# Define the specific words we want to examine
words_of_interest <- c("rabbit", "she", "little", "chortled", "shed",
                       "the", "hoe", "slithy", "galumph")

# Create a table containing only the selected words
# - select() keeps the writer column and the selected words
# - any_of() selects only the columns listed in words_of_interest
selected_word_counts <- author_word_counts %>%
  select(writer, any_of(words_of_interest)) # Keep the writer and selected words

# View the resulting table
print(selected_word_counts)
```

```
## # A tibble: 2 x 9
##   writer         rabbit   she little chortled  shed   the   hoe slithy
##   <chr>           <int> <int>  <int>    <int> <int> <int> <int>  <int>
## 1 Beatrix Potter      9     7      6        0     4   240     1      0
## 2 Lewis Carroll       0   523    108        1     1  1605     0      3
```

Even from this simple frequency table we can see clear differences in the authors' vocabularies. Beatrix Potter's most frequently used word is "rabbit," while Lewis Carroll does not use the word

"rabbit" at all. In contrast, Carroll frequently uses words such as "she" and "little," whereas Potter rarely uses either.

Carroll was famously inventive with language, especially in his use of nonsense and portmanteau words. He had a strong preference for whimsical and linguistically playful terms—words that stretch meaning or defy conventional usage—such as "slithy," "chortle," and "galumph," – especially visible in the dreamlike world of *Through the Looking Glass*.[6]

While this manual inspection suggests that tf-idf will yield interesting results, it also demonstrates the limitations of manual analysis. Such inspection depends on knowing which words to look for in advance, whereas tf-idf can identify distinctive terms throughout the corpus at scale. Our manual inspection can also introduce confirmation bias, that is, the tendency to notice evidence that supports our expectations while overlooking evidence that does not. Metrics such as tf-idf address this problem by identifying distinctive words across an entire dataset rather than relying on an analyst's prior assumptions. This does not mean that tf-idf is free from bias; it reflects its own priorities about what makes a word important. For this reason, it is often insightful to apply multiple metrics—such as tf-idf alongside distance measures or Jensen–Shannon Divergence (JSD)—to examine a dataset from different perspectives.

In this chapter, we begin by examining distinctiveness and then turn to distance, comparing the two metrics to illustrate the different insights they provide as well as the strengths and limitations of each metric.

Using Distinctiveness to Compare Two Authors

Distinctiveness measures build upon the premise of our table by scoring the relationship between words and documents across sets of texts. Instead of manually creating a table ourselves, distinctiveness measures can identify words that are characteristic of an author, novel, document, year, or group.

Distinctivness algorithms like tf-idf assign higher values to words that are unique to each author. "Slithy" would be assigned a high score for Lewis Carroll, and "shed" would be assigned a high score for Beatrix Potter because these words are unique to the respective authors. In contrast, the word "the" would be assigned a lower distinctiveness score, as this common word is not unique to either author.

The following table demonstrates this idea at a small scale. The higher values—such as rabbit for Beatrix Potter or slithy for Lewis Carroll—are still small in absolute terms, but are relatively higher than the others in the table and therefore more distinctive.

```
# Compute tf-idf values
tfidf_table <- word_counts %>%
  bind_tf_idf(word, writer, n)

# Keep only the selected words and sort them by author
# - filter() keeps only the selected words
# - arrange() sorts the words by author and tf-idf score
filtered_tfidf <- tfidf_table %>%
  filter(word %in% words_of_interest) %>% # Keep only the selected words
```

```
  arrange(writer, desc(tf_idf)) # Sort by author and tf-idf score

# Create a side-by-side comparison of the two authors
# - tibble() creates a table from the selected words and tf-idf scores
# - rows 1-6 correspond to Beatrix Potter
# - rows 7-12 correspond to Lewis Carroll
combined <- tibble(beatrix_word = filtered_tfidf$word[1:6], # Store Beatrix Potter's words
                   beatrix_tfidf = filtered_tfidf$tf_idf[1:6], # Store Beatrix Potter's tf-
                   carroll_word = filtered_tfidf$word[7:12], # Store Lewis Carroll's words
                   carroll_tfidf = filtered_tfidf$tf_idf[7:12]) # Store Lewis Carroll's tf-

# Display results
combined %>%
  kable(digits = 6, # show up to 6 decimal points
        format.args = list(scientific = FALSE), # make sure words are in decimal notation,
        col.names = c("Beatrix Word", "tf-idf", # Give columns legible names
                      "Lewis Carroll Word", "tf-idf"))
```

Beatrix Word	tf-idf	Lewis Carroll Word	tf-idf
rabbit	0.001547	slithy	0.000072
hoe	0.000172	chortled	0.000024
little	0.000000	little	0.000000
she	0.000000	she	0.000000
shed	0.000000	shed	0.000000
the	0.000000	the	0.000000

Let's interpret the results. Although "she" appears much more often in Lewis Carroll than in Beatrix Potter, tf-idf assigns it a score of zero because the word appears in both authors' texts. This illustrates a limitation of tf-idf when applied to a very small corpus. With only two documents, any word that appears in both documents receives an inverse document frequency (idf) of zero, causing its tf-idf score to be zero regardless of differences in frequency. However, even with this limitation we can see that tf-idf was successful in measuring whether a word was unique to a document. The word "rabbit," on the other hand, appears only in Beatrix Potter's text. Because it occurs in one document but not the other, it receives a positive idf value and therefore a tf-idf score above zero

For this analysis, the actual values (e.g., 0.001547 versus 0.000072) are less important than their relative ranking. What matters is that rabbit is much more distinctive for Potter than any of the words shown for Carroll are for Carroll within this corpus.

In larger corpora, tf-idf does not reduce all shared words to zero. Instead, words that appear in many documents receive lower scores, while words that appear in relatively few documents receive higher scores. This allows tf-idf to represent varying degrees of distinctiveness rather than treating distinctiveness as a binary property.

We can use distinctiveness to quickly identify the words that appear frequently in Carroll's book but not Potter's. We might expect the results to include a list of words that includes "Jabberwock" (a fearsome creature), the "Snark" (an elusive entity), the "Bandersnatch" (a fast and dangerous

beast), and other terms from his peculiar universe. Indeed, many of these words have rarely, if ever, appeared in publications not authored by Carroll himself.

```
# Keep Lewis Carroll's most distinctive words
# - filter() keeps only Lewis Carroll's words
# - arrange() sorts the words from highest to lowest tf-idf score
# - select() keeps only the word and tf-idf columns
distinctively_carroll <- tfidf_table %>%
  filter(writer == "Lewis Carroll") %>% # Keep only Lewis Carroll's words
  arrange(desc(tf_idf)) %>% # Sort by tf-idf score
  select(word, tf_idf) # Keep only the word and tf-idf columns

# Display the 15 most distinctive words used by Lewis Carroll
distinctively_carroll %>%
  head(15) %>%
  kable(caption = "Lewis Carroll's Most Distinctive Words when Alice in Wonderland is compa
        col.names = c("Word", "tf-idf"),
        escape = FALSE)
```

Table 6.2: Lewis Carroll's Most Distinctive Words when Alice in Wonderland is compared with Peter Rabbit

Word	tf-idf
alice	0.0107534
queen	0.0044024
like	0.0029349
me	0.0022132
again	0.0018764
herself	0.0018043
red	0.0016840
king	0.0014915
knight	0.0014193
cried	0.0013472
dumpty	0.0013231
humpty	0.0013231
here	0.0012750
two	0.0012510
moment	0.0010344

Interestingly, the Jabberwock, Bandersnatch, and Snark are outranked in this comparison by relatively simple characters – Alice, the Queen, the Red Knight, and Humpty-Dumpty – because none of these characters appear in *Peter Rabbit*, while all of those characters are mentioned more than the Jabberwock.

If we were to compare *Through the Looking Glass* with a broader corpus of children's writing, we would get different results. Then, words like "queen," "knight," and even "Humpty-Dumpty" might appear outside of Carroll's corpus, but "Bandersnatch" is likely to remain distinctively Carroll-esque.

Using Distance to Compare Two Authors

Distance provides another metric that we can add to our analytical toolbox. First, we can measure the distance between individual words, allowing us to compare how different metrics emphasize different linguistic patterns within a dataset. Measuring distance in conjunction with tf-idf can reveal the assumptions and limitations of both methods since we can compare the two and see how different measures emphasize different linguistic patterns.

Second, distance allows us to ask broader questions. Rather than focusing only on which terms characterize a particular author, we can measure how linguistically similar or different authors, books, debates, or other collections of texts are from one another. In this way, we will use distance to shift the focus from individual words–like we found tf-idf was particularly useful for–to relationships between entire texts, helping us identify patterns of difference and change across a corpus.

To measure distance we begin with word-frequency, as we did above. Next, we apply a distance measure, JSD, to the word counts instead of tf-idf.

```
# Calculate Jensen-Shannon divergence using our tokens from above
# comparing the word distributions of Lewis Carroll and Beatrix Potter
output <- jsd(word_counts,
              group = "writer", # column containing group labels
              group_list = c("Lewis Carroll", "Beatrix Potter")) %>%
  rename(jsd = 2) # rename the second column to JSD

# Display the 15 words with the highest JSD scores
output %>%
  arrange(desc(jsd)) %>% # Sort by JSD score from highest to lowest
  slice_head(n = 15) %>% # Keep the top 15 words
  kable(caption = "Top 15 Words with the Highest JSD Scores",
        digits = 6, # Display values to six decimal places
        format.args = list(scientific = FALSE), # Use decimal notation instead of scientifi
        col.names = c("Word", "JSD Score")) # Label the output columns
```

Table 6.3: Top 15 Words with the Highest JSD Scores

Word	JSD Score
project	0.010607
gutenberg	0.010483
said	0.007402
i	0.006892
or	0.006188
she	0.005695
work	0.005054
works	0.003449
illustration	0.003347
it	0.002764
e	0.002447

Word	JSD Score
any	0.002350
of	0.002190
her	0.002094
so	0.001866

At first glance, these results look very different from tf-idf. While tf-idf identifies words that are distinctive because they are representative of a particular dataset but not another, JSD identifies words whose relative frequencies differ substantially between two corpora. A higher JSD score indicates that a word contributes more to the divergence between the corpora. The results include the word "she," which we also identified as distinctive. However, they also include common function words such as "any," "so," "it," and "of," which carry little descriptive meaning on their own.

We can place the JSD results alongside the tf-idf results to compare what each metric highlights.

JSD emphasizes words that occur often enough to meaningfully contribute to the overall difference between texts and whose frequencies differ between those texts. A word that appears only once is unlikely to have much influence on JSD because it contributes very little to the overall distribution of words.

As a result, tf-idf tends to identify rare, distinctive terms, whereas JSD often highlights more common words whose usage patterns differ between texts. In this example, JSD returned several high-frequency words, such as "I," "she," "it," and "so," because differences in how often these words are used help distinguish the books.

For this particular analysis, however, measuring distance at the level of each individual word is less informative from a historical standpoint because the results highlight common function words rather than words that reveal clear narrative differences between the authors.

```r
# Keep the 15 most distinctive words according to tf-idf
# - arrange() sorts the words from highest to lowest tf-idf score
# - distinct() keeps only the highest-scoring instance of each word
# - slice_head() keeps the first 15 rows
# - select() keeps only the word and tf-idf columns
top_tfidf <- tfidf_table %>%
  arrange(desc(tf_idf)) %>% # Sort by tf-idf score
  distinct(word, .keep_all = TRUE) %>% # Keep the highest-scoring instance of each word
  slice_head(n = 15) %>% # Keep the top 15 words
  select(word, tf_idf) # Keep only the word and tf-idf columns

# Keep the 15 most distinctive words according to JSD
# - arrange() sorts the words from highest to lowest JSD score
# - slice_head() keeps the first 15 rows
# - select() keeps only the word and JSD columns
top_jsd <- output %>%
  arrange(desc(jsd)) %>% # Sort by JSD score
  slice_head(n = 15) %>% # Keep the top 15 words
  select(word, jsd) # Keep only the word and JSD columns
```

```r
# Create a side-by-side comparison of tf-idf and JSD
# - tibble() creates a table from the top words and scores
comparison_table <- tibble(tfidf_word = top_tfidf$word, # Store the tf-idf words
                           tfidf_score = top_tfidf$tf_idf, # Store the tf-idf scores
                           jsd_word = top_jsd$word, # Store the JSD words
                           jsd_score = top_jsd$jsd) # Store the JSD scores

# Display the comparison table
# - kable() formats the table
# - add_header_above() groups the columns under tf-idf and JSD
comparison_table %>%
  kable(caption = "Top tf-idf and JSD Terms", # Add a table caption
        digits = 6, # Display six decimal places
        format.args = list(scientific = FALSE), # Turn off scientific notation
        col.names = c("Word", "Score", "Word", "Score")) %>% # Label the columns
  add_header_above(c("tf-idf" = 2, "JSD" = 2)) # Group the columns
```

Table 6.4: Top tf-idf and JSD Terms

tf-idf		JSD	
Word	Score	Word	Score
alice	0.010753	project	0.010607
peter	0.004812	gutenberg	0.010483
electronic	0.004640	said	0.007402
queen	0.004402	i	0.006892
foundation	0.003781	or	0.006188
terms	0.003781	she	0.005695
copyright	0.003437	work	0.005054
states	0.003266	works	0.003449
agreement	0.003094	illustration	0.003347
license	0.003094	it	0.002764
like	0.002935	e	0.002447
donations	0.002578	any	0.002350
united	0.002578	of	0.002190
trademark	0.002406	her	0.002094
archive	0.002234	so	0.001866

JSD is often more useful for comparing entire texts than for examining individual words. Instead of measuring the contribution of single words, we will measure the distance between complete books. To make the comparison more meaningful, we will add a second Lewis Carroll book, *Alice's Adventures in Wonderland*, and measure the distance between books.[7] We would expect Alice's Adventures in Wonderland and Peter Rabbit to have a larger distance than Alice's Adventures in Wonderland and Through the Looking-Glass, since the two Carroll books share more stylistic and lexical features.

```r
# URL for Alice's Adventures in Wonderland from Project Gutenberg
alice_url <- "https://www.gutenberg.org/files/11/11-0.txt"

# Read the text file line by line into R
alice_text <- readLines(alice_url, encoding = "UTF-8", warn = FALSE)

alice_df <-   tibble(writer = "Lewis Carroll",
                     booktitle = "Alice's Adventures in Wonderland",
                     text = alice_text)

# Combine the three books into one data frame
combined_books <- bind_rows(combined_books, alice_df)

# Count the words in each book
# - unnest_tokens() tokenizes the text into one word per row
# - filter() keeps only alphabetic words
# - count() counts the occurrences of each word in each book
book_counts <- combined_books %>%
  unnest_tokens(word, text) %>% # Break the text into words
  filter(str_detect(word, "^[a-z]+$")) %>% # Keep only alphabetic words
  count(booktitle, word) # Count words by book
```

Now that we have our tokenized words and word count by book, we can apply a different metric from `dhmeasures`, called `original_jsd()`, to find the distance between books. In this context, a lower distance value indicates greater linguistic similarity, while a higher value indicates greater difference.

This code calculates the overall Jensen–Shannon divergence (JSD) between each pair of three books and then formats the results into a table.

The two Lewis Carroll books, *Alice's Adventures in Wonderland* and *Through the Looking Glass*, are notably more similar to one-another.

```r
# Calculate the pairwise JSD distances between the books
# - original_jsd() computes the Jensen-Shannon distance for each pair of books
book_distances <- original_jsd(book_counts,
                               group = "booktitle", # Group the data by book title
                               group_list = c("Alice's Adventures in Wonderland",
                                              "Peter Rabbit",
                                              "Through the Looking Glass")) # Compare these

# Create a table of the pairwise JSD distances
# - tibble() creates a table from the JSD results
distance_table <- tibble(Comparison = c("Alice vs Peter", # Compare Alice and Peter Rabbit
                                        "Alice vs Looking Glass", # Compare Alice and Looki
                                        "Peter vs Looking Glass"), # Compare Peter Rabbit a
                         JSD = as.numeric(book_distances[1, ])) # Store the JSD distances
```

```r
# Display results
distance_table %>%
  kable(digits = 6,
        format.args = list(scientific = FALSE),
        col.names = c("Comparison", "JSD Distance"))
```

Comparison	JSD Distance
Alice vs Peter	0.441919
Alice vs Looking Glass	0.120065
Peter vs Looking Glass	0.447363

Using Distance to Analyze A Larger Corpus

Now, what if we had access to a larger repository of books? Instead of comparing two specific authors, we might wish to learn which writers use vocabularies similar to Lewis Carroll's. For instance, we could search for books with a comparable ratio of playful or invented words to conventional ones.

We might be interested in identifying the top ten authors who make frequent use of neologisms or poetic nonsense. If we were doing distance comparisons on a larger set, Carroll might score closer to authors like Edward Lear, Roald Dahl, or Spike Milligan, who also play with absurdity and sound, and farther from Beatrix Potter, whose style is more naturalistic. Lear's texts are on Project Gutenberg, so we can import them into the R environment. For this example, we will download Lear's *Nonsense Books* as well as Kenneth Grahame's *The Wind in the Willows*, Frank Baum's *The Wonderful Wizard of Oz*, and Rudyard Kipling's *The Jungle Book*.

```r
# By Lewis Carroll
alice_text <- readLines("https://www.gutenberg.org/files/11/11-0.txt",
                        encoding = "UTF-8",
                        warn = FALSE)

# By Lewis Carroll
looking_glass_text <- readLines("https://www.gutenberg.org/files/12/12-0.txt",
                                encoding = "UTF-8",
                                warn = FALSE)

# By Beatrix Potter
peter_text <- readLines("https://www.gutenberg.org/files/14838/14838-0.txt",
                        encoding = "UTF-8",
                        warn = FALSE)

# By Kenneth Grahame
wind_text <- readLines("https://www.gutenberg.org/files/289/289-0.txt",
                       encoding = "UTF-8",
                       warn = FALSE)
```

```
# By L. Frank Baum
wizard_text <- readLines("https://www.gutenberg.org/files/55/55-0.txt",
                         encoding = "UTF-8",
                         warn = FALSE)

# By Rudyard Kipling
jungle_text <- readLines("https://www.gutenberg.org/files/236/236-0.txt",
                         encoding = "UTF-8",
                         warn = FALSE)

# By Edward Lear
lear_text <- readLines("https://www.gutenberg.org/files/13650/13650-0.txt",
                       encoding = "UTF-8",
                       warn = FALSE)

# Combine the books into a single table
# - bind_rows() combines the books into one dataset
books <- bind_rows(tibble(book = "Alice's Adventures in Wonderland", text = alice_text), #
                   tibble(book = "Through the Looking-Glass", text = looking_glass_text), #
                   tibble(book = "The Tale of Peter Rabbit", text = peter_text), # Add Pete
                   tibble(book = "The Wind in the Willows", text = wind_text), # Add The Wi
                   tibble(book = "The Wonderful Wizard of Oz", text = wizard_text), # Add T
                   tibble(book = "The Jungle Book", text = jungle_text), # Add The Jungle B
                   tibble(book = "Nonsense Books", text = lear_text)) # Add Nonsense Books

# Count the words in each book
# - unnest_tokens() tokenizes the text into one word per row
# - filter() keeps only alphabetic words
# - count() counts the occurrences of each word in each book
book_counts <- books %>%
  unnest_tokens(word, text) %>% # Break the text into words
  filter(str_detect(word, "^[a-z]+$")) %>% # Keep only alphabetic words
  count(book, word) # Count words by book

# Create a list of the books to compare with Alice's Adventures in Wonderland
comparison_books <- c("Through the Looking-Glass",
                      "The Tale of Peter Rabbit",
                      "The Wind in the Willows",
                      "The Wonderful Wizard of Oz",
                      "The Jungle Book",
                      "Nonsense Books")
```

Now we can iterate through each book and compare it to *Alice's Adventures in Wonderland*.

```
# Create a table to store the JSD distances
distances <- tibble()
```

```
# Compare Alice's Adventures in Wonderland to each book
# - for() repeats the comparison for each book
# - original_jsd() calculates the Jensen-Shannon distance
# - bind_rows() adds each result to the table
for(book in comparison_books) {

  # Calculate the JSD between Alice and the current book
  score <- original_jsd(book_counts,
                        group = "book", # Group the data by book
                        group_list = c("Alice's Adventures in Wonderland", book)) # Compare

  # Add the result to the table
  distances <- bind_rows(distances,
                         tibble(comparison_book = book, # Store the comparison book
                                jsd_distance = score[[1]])) } # Store the JSD distance

# Show the results in decimal notation
distances %>%
  arrange(jsd_distance) %>%
  kable(digits = 6,
        format.args = list(scientific = FALSE),
        col.names = c("Book", "JSD Distance from Alice in Wonderland"))
```

Book	JSD Distance from Alice in Wonderland
Through the Looking-Glass	0.120067
The Wonderful Wizard of Oz	0.209991
The Wind in the Willows	0.213553
The Jungle Book	0.248262
Nonsense Books	0.293408
The Tale of Peter Rabbit	0.408177

Analysis. Jo?

As a takeaway, distance is useful for quantifying how far apart texts are in terms of their overall linguistic patterns. Variants of distance measurements could make it another alternative for pinpointing the specific features (like “chortle” or “jabberwock”) that make a text unique – but in general, we leave this task to distinctiveness (for more on the nuances of each approach, please see Guldi’s *The Dangerous Art of Text Mining*).[8]

Distinctiveness Applied to History

Both of these methods of measurement invite a broader reflection on computational methods. The ways in which we decide to process and analyse data—whether through distance or distinctiveness—are shaped by the questions we pose. The computational steps involved in data processing are as much a part of knowledge creation as the act of interpretation. The methods we

choose mediate our access to historical records, and their respective affordances and limitations may lead us to shift our analytical focus—for instance, from emphasizing distinctiveness as an indicator of uniqueness in a corpus, to using distance to reveal relational patterns, or to adopting an entirely different metric more suitable to the historical material at hand. Both methodological decisions are interpretive acts in their own right, as they shape how we understand and conceptualize the data before us.

As we will show, both tf-idf and JSD support anaysts' interpretations of history, allowing us to answers questions like: which words are characteristic of one speaker and not another? Which words are most symptomatic of the 1860s, a decade characterized by an economic crisis related to the cotton industry as well as labor disputes, and what can we learn about the 1860s from applying methods of distance or divergence?

Here are some examples for how an analyst might think through the lens of distinctiveness when working with the Hansard corpus:

- An analyst might use distinctiveness measures to identify words that were frequently used by Gladstone but not by other prime ministers. The results could highlight the linguistic patterns, priorities, or recurring concerns that made Gladstone a distinctive political figure. For example, if Gladstone consistently used words related to reform more often than others, Gladstone's discourse might reveal key characteristics of his political or rhetorical strategy.
- An analyst might apply distinctiveness to find out the words used by Gladstone early in his career, but not later, and later in his career, but not earlier.
- An analyst could use distinctiveness measures to analyze political parties and the words they used that were excluded from use by other parties at the time. By comparing the vocabulary used by members of different parties, the analyst might uncover terms that were uniquely associated with one party's rhetoric and thereby gain insight into the rhetorical strategies emphasized by each group. Our analytic approach can help trace how party identities were constructed and communicated through language.

Here are some examples for how an analyst might think through the lens of distance when working with Hansard:

- An analyst might use distance to see which speakers' lexicons most resemble one another.
- An analyst might use distance to compare units of time – weeks, months, or years – with all other units of weeks, months, or years – in order to answer the question, "when were new ideas and concerns coming to parliament's attention? In *The Dangerous Art of Text Mining*, Guldi explains how distinctiveness can be used to investigate the most distinctive years, months, weeks, and days of any century, with the end of highlighting exceptional conversations that are unlike the day-to-day business that is the matter of any institution.[9]

Shifting to the Hansard corpus, we will first demonstrate tf-idf to compare multiple decades back-to-back to surface the words that are most unique to one decade and not the others. We will then use JSD to explore the lexical patterns of debates surrounding the use of the word "cotton" measured against other decades. The results will demonstrate how changes in the way in which difference is measured reveal different dimensions of a corpus. We will encourage readers to reflect on these differences, using them as a way of expanding their insight into the nature of difference, the use of comparable algorithms as different tools in a process of analysis, and the beginning of a process of critically thinking about data, algorithm, and historical truth.

As these examples indicate, one of the most powerful ways of applying either metric, for the historian, is about understanding change over time.

Distinctiveness as an Approach to Change Over Time

Distinctiveness is a powerful concept for understanding historical change because it allows the analyst to understand which words are most unique to any given political moment in time.

If the word "cotton" appears frequently in the years 1860-69, but cotton is almost never mentioned in the years 1806-1859 or 1970-1911, then "cotton" can be considered distinctive of the 1860s. If the word "cotton" appears with a similar frequency in speeches from each decade in the nineteenth century, it is not distinctive of any given decade but rather a reoccurring concern throughout each decade.

Measuring the Distinctiveness of Debates Speeches from the 1830s and 1860s using tf-idf

We will begin our work on Hansard by measuring the distinctiveness of language by decade from 1830 through 1860. The code that follows will use distinctiveness to compare the words used by parliamentary speakers in the 1830s against the 1860s.

First we will load our libraries and our data, which includes the `stop_words` data from `tidytext` and the 1830 and 1860 Hansard debates. We will then use a version of tf-idf from `tidytext` similar to how we did above. To track the association of words with their respective decades, we will add a column named "decade" and assign each entry to its corresponding decade.[10]

We can use `bind_rows()` to combine `hansard_1830` and `hansard_1860` into a new data frame by stacking them vertically. Stacking two data frames differs from using a join function such as `left_join()` or `inner_join()`, which merge data frames by matching rows based on common columns. `bind_rows()` does not merge columns; it instead appends the rows of one data frame to the other, even if their column names do not fully align.

```
# Load tidytext's built-in list of stop words for filtering out common words
data("stop_words")

# Load the Hansard debates from the 1830s and 1860s
data("hansard_1830")
data("hansard_1860")

# Add a decade label to each Hansard dataset
# - mutate() creates a new decade column
hansard_1830 <- hansard_1830 %>%
  mutate(decade = 1830) # Label all rows as 1830

hansard_1860 <- hansard_1860 %>%
  mutate(decade = 1860) # Label all rows as 1860
```

```r
# Combine the two Hansard datasets
# - bind_rows() stacks the rows from both datasets into one table
hansard_data <- bind_rows(hansard_1830, hansard_1860) # Combine the two datasets

# Inspect the data
hansard_data[, 2:3] %>%
  sample_n(5) %>%
  kable(caption = "Sample from Hansard Data")
```

Table 6.7: Sample from Hansard Data

text	decade
Mr. Weguelin said his father had been Member for Southampton, which cost £2, 300, and, therefore, he thought Youghal would cost £1, 000; but by August he had spent £1, 500, and by the election £5, 000.	1860
In the navy many entertain a preference for the Whitworth gun, which they consider is more simple than the Armstrong or the stud; in the army a great majority of officers are in favour of the Armstrong or the stud system.	1860
Member for Norfolk (Mr. Bentinck) related to the comparison between hammered and rolled iron.	1860
I have no reason to believe in its existence, and for many reasons I am inclined to doubt it; still I could not take upon myself to affirm that some sort of engagement or understanding respecting Savoy does not exist between the two Governments. ”	1860
But that measure was to be confined to railways which were thenceforward to be constructed, and it did not extend to railways which had already been completed.	1860

We can now process our dataset to prepare it for tf-idf analysis. The following code applies a series of transformations: it tokenizes the text into a one-word-per-row format, removes stop words using the predefined list from the `tidytext` package, and filters out digits using a regular expression. While chaining multiple functions together, we ensure that the data is cleaned and structured appropriately for tf-idf calculations.

Since the following code applies multiple functions at once to a larger subset of decades, it may take longer to finish running.

Next, we prepare a table of counts that are necessary for the distinctiveness algorithm: * _words_per_decade_, a count of how many times each individual word was used per decade

From this table, the tf-idf function will be able to calculate the various components fo the tf-idf algorithm, including: * the number of total unique words for the 1830s * the number of total unique words for the 1860s * the number of total unique words for the corpus overall

Sidenote: Minimizing the Impact of Data Messiness

For the sake of our analysis, we are only keeping words that appear more than once per decade–an operation that we perform with `filter(n > 1)`. One reason we make this choice is because words

appearing only once in the 19th-century Hansard corpus are often the result of OCR errors. When the transcripts were digitized, poor image quality resulted in a mismatch between the digitized word and the word as it was recorded on the transcript. The letter "O," for example, may have been interpreted as the number zero, or the letter "h" might have been interpreted as two "i's" such as "ii." Future versions of the digital Hansard corpus may address OCR errors.

In the meantime, however, analysts often have to make judgment calls that entail narrowing a corpus to words, whether to eliminate OCR errors or simply to identify more common words. Tweaking the value of the number x in the expression `filter(n > x)` will alter the final results of this chapter in ways that may be meaningful by removing more low frequency words.

```
# Split the Hansard text into individual words
# - unnest_tokens() tokenizes the text into one word per row
# - anti_join() removes common stop words
# - filter() removes words containing digits
tokenized_hansard_data <- hansard_data %>%
  unnest_tokens(word, text) %>% # Break the text into words
  anti_join(stop_words) %>% # Remove stop words
  filter(!str_detect(word, "[0-9]")) # Remove words containing digits
```

```
# Count the words used in each decade
# - count() counts the occurrences of each word by decade
# - filter() removes words that appear only once
words_per_decade <- tokenized_hansard_data %>%
  count(decade, word, sort = TRUE, name = "word_count_per_decade") %>% # Count words by dec
  filter(word_count_per_decade > 1) # Keep words that appear more than once

# Display a sample of the word counts
# - rename() gives the columns descriptive names
# - sample_n() selects a random sample of rows
# - arrange() sorts the rows by word count
# - pivot_wider() reshapes the table into a wide format
words_per_decade %>%
  rename(hansard_word = word, # Rename the word column
         hansard_decade = decade) %>% # Rename the decade column
  sample_n(8) %>% # Select eight random rows
  arrange(desc(word_count_per_decade)) %>% # Sort by word count
  pivot_wider(names_from = hansard_word, # Create one column per word
              values_from = word_count_per_decade, # Fill the columns with word counts
              values_fill = 0) # Replace missing values with zeros
```

```
## # A tibble: 2 x 9
##   hansard_decade liability credits  tack satellites cottier  pute  lawn
##            <dbl>     <int>   <int> <int>      <int>   <int> <int> <int>
## 1           1830       314       0     0         12       0     4     0
## 2           1860         0      92    52          0       8     0     2
## # i 1 more variable: pronunciation <int>
```

The results are ready to be passed to the distinctiveness algorithm so that the tf-idf metric can be computed for each word-decade pair.

We use the function `bind_tf_idf()` to calculate distinctiveness for any word-document pair, using the arguments *word*, *document*, and *n*. To calculate distinctiveness by decade, we set the argument *document* as "decade."

```
# Calculate the tf-idf score for each word by decade
# - bind_tf_idf() calculates the tf-idf score from the word counts
hansard_tf_idf <- words_per_decade %>%
  bind_tf_idf(word, decade, word_count_per_decade) # Calculate tf-idf scores
```

```
# View the top 5 results
hansard_tf_idf %>%
  arrange(desc(tf_idf)) %>%
  sample_n(5)
```

```
## # A tibble: 5 x 6
##   decade word       word_count_per_decade          tf   idf      tf_idf
##    <dbl> <chr>                      <int>       <dbl> <dbl>       <dbl>
## 1   1860 flood                         94 0.00000979  0     0
## 2   1830 debartsch                      2 0.000000221 0.693 0.000000153
## 3   1830 granite                        5 0.000000553 0     0
## 4   1830 hungerford                    16 0.00000177  0     0
## 5   1830 furnished                   1181 0.000131    0     0
```

Please note that the output of `bind_tf_idf()` includes the component calculations we mentioned: - one row for each decade in which each word appears - the total number of times each word appears in this decade (given in `word_count_per_decade`, the product of our calculation to produce the `words_per_decade` data frame) - the "term frequency" calculation, a metric of how often each word appears per decade, normalized against the total number of words in each decade. A high TF means that the word appears frequently in a given decade. - the "inverse document frequency" metric, a calcuation of how uncommon the word is across the entire corpus. A high IDF means that a word is rare across the corpus. - the tf-idf score – the most important score for interpreting the results. A high tf-idf score means that a word is highly distinctive of a decade, i.e., it scarcely appears in the other decade(s) of a corpus.

Now that we have a tf-idf metric for each word-decade pair, we can use these scores to interpret the differences between the 1830s and 1860s.

```
# Keep the 15 most distinctive words from each decade
# - group_by() groups the words by decade
# - slice_max() keeps the highest tf-idf words in each decade
# - arrange() sorts the results by decade and tf-idf score
# - ungroup() removes the grouping
top_hansard_tf_idf <- hansard_tf_idf %>%
  group_by(decade) %>% # Group by decade
```

```
  slice_max(tf_idf, n = 15) %>% # Keep the top 15 words
  arrange(decade, desc(tf_idf)) %>% # Sort by decade and tf-idf score
  ungroup() # Remove the grouping
```

Let's inspect the data.

```
# Display the most distinctive words from each decade
# - group_by() groups the words by decade
# - slice_max() keeps the top 10 words in each decade
# - arrange() sorts the results by decade and tf-idf score
# - mutate() rounds the tf-idf scores
# - select() keeps only the relevant columns
# - kable() formats the results as a table
top_hansard_tf_idf %>%
  group_by(decade) %>% # Group by decade
  slice_max(tf_idf, n = 10) %>% # Keep the top 10 words
  arrange(decade, desc(tf_idf)) %>% # Sort by decade and tf-idf score
  mutate(tf_idf = round(tf_idf, 6)) %>% # Round the tf-idf scores
  select(decade, word, tf_idf) %>% # Keep the relevant columns
  kable(col.names = c("Decade", "Word", "tf-idf Score"), # Label the columns
        digits = 6, # Display six decimal places
        format.args = list(scientific = FALSE), # Turn off scientific notation
        caption = "Top 10 tf-idf Words for the 1830s and 1860s") # Add a table caption
```

Table 6.8: Top 10 tf-idf Words for the 1830s and 1860s

Decade	Word	tf-idf Score
1830	miguel	0.000073
1830	gosford	0.000036
1830	pedro	0.000031
1830	terceira	0.000027
1830	benefitted	0.000023
1830	mulgrave	0.000020
1830	vigors	0.000018
1830	donna	0.000017
1830	wetherell	0.000016
1830	boroughbridge	0.000015
1860	disestablishment	0.000062
1860	schleswig	0.000059
1860	churchward	0.000057
1860	fenian	0.000048
1860	turret	0.000042
1860	plated	0.000041
1860	disendowment	0.000040
1860	cairns	0.000040

Decade	Word	tf-idf Score
1860	newdegate	0.000040
1860	disraeli	0.000039

Words that were distinctive in the 1830s, compared to later decades, often reference the Carlist Wars—a series of civil wars in Spain that began in 1833 over the dispute regarding the rightful successor to the throne. Portugal, France and the United Kingdom supported the regency, and even intervened in the conflict by sending forces to confront the Carlist army. Subsequently, the reference to "Carlist" can possibly explain the number of Spanish names/words listed in the bar plot, including "Miguel," "Pedro," and "Terceira."

Other terms that appear as distinctive of the 1830s in comparison to the 1860s are references to individuals and places whose names punctuated routine debates of the period. The year 1831 marked the death of the 1st Earl of Mulgrave, a distinguished politician who had served as Foreign Secretary and was subsequently memorialized in many speeches. From 1830-2, Charles Wetherell represented Boroughbridge in Parliament, explaining the inclusion of his name and the constituency he served. In 1839 the recently colonized city of Gosford, in New South Whales, Australia, is given its name after the 2nd Earl of Gosford, a friend of the governor of New South Whales.

Words distinctive of the 1860s include several references to contemporary matters of foreign affairs unique to the decade, for instance, "schleswig," a reference to the Second Schleswig War (1864), wherein Germany and Denmark commenced an altercation over disputed territory that included the province of Schleswig-Hollsteine. "Fenian" references the 1867 Fenian Uprising, a failed rebellion against British rule in Ireland. The name "Disraeli" marks the prime minister who oversaw the 1867 Reform Act that enfranchised much of Britain's working class. The term "Crimean" references the Crimean War, which broke from 1853-56. "Sewage" references contemporary questions of the rebuilding of London's water supply.

This simple exercise demonstrates tf-idf in action, providing a concrete example of how the method identifies terms that are unique to each document, with minimal overlap between them.

This toy analysis is limited in implication, however, because historians rarely want to compare two distant periods of time. Far more instructive will be the results of analyzing each decade between 1830 to 1860 against each other.

Analyzing the Distinctiveness of Decades Over a Half-Century, from 1830 to 1870

The following code loads the 1840 and 1850 Hansard debates and combines them into our dataframe that will be used to measure tf-idf.

```
# Load Hansard data for the 1840s and 1850s
data("hansard_1840")
data("hansard_1850")

# Add a decade label to each Hansard dataset
# - mutate() creates a new decade column
```

```r
hansard_1840 <- hansard_1840 %>%
  mutate(decade = 1840) # Label all rows as 1840

hansard_1850 <- hansard_1850 %>%
  mutate(decade = 1850) # Label all rows as 1850

# Combine the Hansard datasets
# - bind_rows() stacks the rows from each dataset into one table
hansard_data_1830_to_1870 <- hansard_data %>%
  bind_rows(hansard_1840) %>% # Add the 1840s data
  bind_rows(hansard_1850) # Add the 1850s data
```

In preparation for using tf-idf across multiple decades, we use the following codes to generate word frequencies in text data across specific time periods (e.g., decades). It uses a `for()` loop to iterate through each decade for preprocessing. It preprocesses the text by tokenizing it into words, removing stop words, and filtering out irrelevant tokens (like numbers), then counts the most frequently used words for each decade. The results are stored in a new data frame, `hansard_count`.

While a `for` loop is not needed to perform these actions, we have chosen to use one here to process text data separately for each decade in the dataset. Otherwise, tokenizing all four decade subsets simultaneously could exceed available computer memory. The code may run slow on older machines – give it time to run.

```r
# Define the decades to analyze
decades <- c(1830, 1840, 1850, 1860)

# Create an empty table to store the word counts
# - data.frame() creates an empty table
hansard_words_1830_to_1870 <- data.frame()

# Count the words in each decade
# - for() repeats the analysis for each decade
for (d in decades) {

  # Keep only the speeches from the current decade
  # - filter() keeps rows matching the current decade
  new_decade <- hansard_data_1830_to_1870 %>%
    filter(decade == d) # Keep only the current decade

  # Split the text into individual words
  # - unnest_tokens() tokenizes the text into one word per row
  # - anti_join() removes common stop words
  # - filter() removes words containing digits
  tokenized_new_decade <- new_decade %>%
    unnest_tokens(word, text) %>% # Break the text into words
    anti_join(stop_words) %>% # Remove stop words
    filter(!str_detect(word, "[:digit:]")) # Remove words containing digits
```

```
  # Count the words used in the current decade
  # - count() counts the occurrences of each word
  # - filter() keeps words appearing more than 20 times
  new_decade_count <- tokenized_new_decade %>%
    count(decade, word, sort = TRUE) %>% # Count words by decade
    filter(n > 20) # Keep words appearing more than 20 times

  # Add the current decade's word counts to the results
  # - bind_rows() appends the new rows
  hansard_words_1830_to_1870 <- bind_rows(hansard_words_1830_to_1870,
                                          new_decade_count) } # Append the results
```

We now have a dataset, hansard_count, which includes each word along with the number of times it appears in a given decade.

```
# Display a sample of the most frequent words by decade
# - group_by() groups the words by decade
# - slice_max() keeps the two most frequent words in each decade
# - arrange() sorts the results by decade and word count
hansard_words_1830_to_1870 %>%
  group_by(decade) %>% # Group by decade
  slice_max(n, n = 2) %>% # Keep the two most frequent words
  arrange(desc(decade), desc(n)) # Sort by decade and word count
```

```
## # A tibble: 8 x 3
## # Groups:   decade [4]
##   decade word       n
##    <dbl> <chr>  <int>
## 1   1860 hon   125781
## 2   1860 house 113329
## 3   1850 hon   125465
## 4   1850 house 118861
## 5   1840 hon   135879
## 6   1840 house 115275
## 7   1830 house 125218
## 8   1830 hon   121494
```

We can now apply tf-idf and visualize our results, in this case, the first 15 words that appear for each decade.

```
# Find the most distinctive words in each decade
# - bind_tf_idf() calculates the tf-idf score for each word
# - group_by() groups the words by decade
# - slice_max() keeps the top 15 words in each decade
most_dist_hansard_words <- hansard_words_1830_to_1870 %>%
  bind_tf_idf(word, decade, n) %>% # Calculate tf-idf scores
```

```
  group_by(decade) %>% # Group by decade
  slice_max(tf_idf, n = 15) # Keep the top 15 words
```

```
# Display the most distinctive words from each decade
# - group_by() groups the words by decade
# - slice_max() keeps the top 10 words in each decade
# - ungroup() removes the grouping
# - arrange() sorts the results by decade and tf-idf score
# - mutate() rounds the tf-idf scores
# - select() keeps only the relevant columns
# - kable() formats the results as a table
most_dist_hansard_words %>%
  group_by(decade) %>% # Group by decade
  slice_max(tf_idf, n = 10) %>% # Keep the top 10 words
  ungroup() %>% # Remove the grouping
  arrange(decade, desc(tf_idf)) %>% # Sort by decade and tf-idf score
  mutate(tf_idf = round(tf_idf, 6)) %>% # Round the tf-idf scores
  select(decade, word, tf_idf) %>% # Keep the relevant columns
  kable(col.names = c("Decade", "Word", "tf-idf Score"), # Label the columns
        digits = 6, # Display six decimal places
        format.args = list(scientific = FALSE), # Turn off scientific notation
        caption = "Top 10 tf-idf Words by Decade") # Add a table caption
```

Table 6.9: Top 10 tf-idf Words by Decade

Decade	Word	tf-idf Score
1830	marquis	0.000139
1830	miguel	0.000074
1830	gosford	0.000072
1830	carlos	0.000062
1830	appleby	0.000057
1830	terceira	0.000055
1830	raphael	0.000050
1830	mulgrave	0.000041
1830	shew	0.000040
1830	aldborough	0.000036
1840	ovans	0.000045
1840	warner	0.000044
1840	o'driscoll	0.000043
1840	sliding	0.000038
1840	gauge	0.000037
1840	stockdale	0.000037
1840	junta	0.000036
1840	ameers	0.000036
1840	mott	0.000035
1840	keane	0.000033

Decade	Word	tf-idf Score
1850	crimea	0.000186
1850	raglan	0.000133
1850	kars	0.000106
1850	sebastopol	0.000100
1850	balaklava	0.000081
1850	sadleir	0.000047
1850	whiteside	0.000046
1850	hebdomadal	0.000044
1850	mather	0.000039
1850	oude	0.000038
1860	disestablishment	0.000127
1860	fenian	0.000098
1860	turret	0.000086
1860	plated	0.000083
1860	savoy	0.000080
1860	garibaldi	0.000062
1860	disestablished	0.000061
1860	abyssinia	0.000059
1860	cased	0.000059
1860	churchward	0.000058

By analyzing the words most characteristic of each decade (the 1830s, 40s, 50s, and 60s), we can easily see a link between those terms and historical events.

Some of the words may key the analyst into rhetorical shifts, like the old-fashioned spelling "shew," discarded after the 1830s. Others attune us to seemingly minor issues that neverthless were discussed frequently in their time. For instance, the distinctive words of the 1830s now include the term "impropriators," a designation of a layperson who held the income rights of a church benefice. The improprietors' rights dated from the Tudor dissolution of the monasteries. Lay impropriators became a target of tithe reform in the 1830s. Other words suggest known turning points of history, for example the governance of railway gauges in the 1840s, the Crimean War in the 1850s, and the prominence of Fenian terrorism and the rifle in the 1860s. Still other words may lead us to new research questions – for instance, asking what is the relevance of "differential" in the 1840s? Our question might lead us to discussions of the "differential duty" discussed by the House of Lords in April 1840, when they mooted making foreign exporters pay higher duties, so as to protect domestic interests tied to monopolies.

Cleaning Out the Cache so as to Preserve Memory

At this stage of our analysis, we are working with several datasets. Working with multiple datasets at a given time can provide us with a richer overview of the historical records, however, it can consume a significant amount of memory on our computer. To free memory, we can remove some of the large datasets from our Global Environment before going to our next examples and analyses.

We can remove data using `rm()` while passing the variables you wish to clear. For example, `rm()` can be passed a single variable name or multiple variable names.

```
# Remove both datasets from the global environment
rm(hansard_1830, hansard_1840)
```

Or, if instead we wish to remove all variables, `rm()` can be passed the output of `ls()`, a function that lists all the variables from the Global Environment.

```
# List all objects currently in the global environment
ls()
```

```
##  [1] "alice_df"                  "alice_text"
##  [3] "alice_url"                 "author_word_counts"
##  [5] "book"                      "book_counts"
##  [7] "book_distances"            "books"
##  [9] "combined"                  "combined_books"
## [11] "comparison_books"          "comparison_table"
## [13] "d"                         "decades"
## [15] "distance_table"            "distances"
## [17] "distinctively_carroll"     "filtered_tfidf"
## [19] "hansard_1850"              "hansard_1860"
## [21] "hansard_data"              "hansard_data_1830_to_1870"
## [23] "hansard_tf_idf"            "hansard_words_1830_to_1870"
## [25] "jungle_text"               "lear_text"
## [27] "looking_glass_df"          "looking_glass_text"
## [29] "looking_glass_url"         "most_dist_hansard_words"
## [31] "new_decade"                "new_decade_count"
## [33] "output"                    "peter_df"
## [35] "peter_text"                "peter_url"
## [37] "score"                     "selected_word_counts"
## [39] "stop_words"                "tfidf_table"
## [41] "tokenized_hansard_data"    "tokenized_new_decade"
## [43] "tokens"                    "top_hansard_tf_idf"
## [45] "top_jsd"                   "top_tfidf"
## [47] "wind_text"                 "wizard_text"
## [49] "word_counts"               "words_of_interest"
## [51] "words_per_decade"
```

```
# List all objects currently in the global environment
all_variables <- ls()

# Remove all objects from the global environment
rm(list = all_variables)
```

Following `rm()` with `gc()` (which stands for "garbage collection") ensures that memory occupied by removed objects is fully reclaimed. When an object is deleted using `rm()`, R does not immediately free up the associated memory; instead, the memory remains allocated until the garbage collector runs. By calling `gc()`, we force R to release unused memory, preventing excessive memory

consumption. Releasing unused memory may be particularly important when working with large datasets or performing memory-intensive operations, as it helps reduce the risk of running out of the computer crashing or being unable to process additional data.

```
# Run garbage collection to reclaim memory
gc()
```

```
##            used  (Mb) gc trigger   (Mb)  max used   (Mb)
## Ncells  2465196 131.7   15148545  809.1  18818723 1005.1
## Vcells 12698129  96.9  489584528 3735.3 636627012 4857.1
```

The output from the `gc()` function provides a snapshot of memory usage and triggers garbage collection. The Ncells store internal structures, like lists, while the Vcells store numerical, character, and logical data. The gc trigger values indicate the memory thresholds that, when exceeded, will automatically initiate garbage collection.

Sometimes, due to memory fragmentation, R may not be able to release memory back to the operating system, even after running `gc()`.Memory fragmentation occurs when R allocates memory in multiple small chunks over the course of a session. As objects are created and removed, chunks of data become scattered throughout the computer's memory. R's garbage collector (`gc()`) can free memory, but it cannot always merge these scattered memory blocks into a single large block that the operating system can reclaim. As a result, memory usage can remain high, even if most objects have been removed.

Memory consumption can be seen in RStudio above the Global Environment panel. If a significant amount of memory remains reserved, it may be necessary to restart R to fully release fragmented memory. We can restart RStudio by closing out of RStudio and without saving the work space, or by running `q()` with the parameter `save` set to "no".

Important: Running the following code block will quit RStudio.

```
# Quit the R session without saving the current workspace
q(save = "no")
```

Distance for Historical Analysis

Distance is used to compare the distributions of words in any two corpora. In general, it is less used to identify the variables that are most characteristic of a corpus, and more frequently used when the analyst needs to position different corpora on a "magic ruler" that ranks them from most similar to least similar to some text.

A historian might use distance for the following applications: * An analyst might choose to identify the speakers whose overall use of political vocabulary most closely resembles Gladstone's. They might then ask whether those speakers were all members of his political party, or whether some came from across the political divide. An analysis could tell us who shared his linguistic style, and whether that style aligned with partisan boundaries or not. * An analyst might use distance to rank the individual speeches that sound the most like any one of a set political passages excerpted from

contemporary novels, including those of Charles Dickens, which frequently engage political phenomena. This process might help the analyst to discover the likeliest speeches that Dickens was attending to when he wrote various novels. * An analyst might compare parliamentary speeches and newspaper stories over time to ask questions about when the lines of influence run from parliament to the newspaper, and when questions are first raised in the newspaper and later in parliament * An analyst might apply party distance by decade to inform research on how party identities shifted over time in response to political events or ideological realignments.

In the sections that follow, we will only address the first of these queries – the matter of how to compare two decades of time to highlight what was coming into parliament's attention and what was fading over a twenty-year period.

The method introduced, however, once mastered, can be applied to all other problems of analysis. Indeed, comparing subcorpora with distance underlies a great many possible programs of analysis, each of which can be accomplished by applying distance measurements to different corpora – that is, to comparing one decade against all other decades, one week to all other weeks, one speaker to all other speakers, and one speech to all other speeches. A great many problems of abstract analysis can be rendered into text-mining problems by recognizing the impulse of comparison at their heart, and translating this basic comparison into distance measurements.

Jenson-Shannon Divergence (JSD)

Any algorithm emphasizes certain aspects of a corpus while de-emphasizing others. Investigating the implications of adopting different algorithms is a critical approach to working with data. To illustrate this point, we introduce another metric for measuring a corpus: a modified form of divergence known as Partial Jensen-Shannon Divergence (Partial JSD). Jensen-Shannon Divergence (JSD) is a measure from information theory that quantifies the similarity between probability distributions by calculating the divergence between their averaged distributions. Unlike other distinctiveness measures, which often highlight exceptional words or phrases between two documents, JSD provides a general framework for comparing textual data by treating documents as probability distributions over words. This approach allows us to use Partial JSD to quantify how "close" or "distant" the language between two datasets are.

Distance is a quantitative measure that captures the degree of similarity or difference in the distribution of a word's occurrence across two or more groups. In text analysis, distance measures assess how consistently or variably a word appears in different datasets, helping to determine whether its usage patterns align or diverge. These measures allow for ranking datasets based on the extent to which words are expressed at comparable rates across them, providing a structured way to compare linguistic patterns, such as by identifying patterns of relative similarity or dissimilarity based on word frequencies, term distributions, or other textual features.

In the following section, we will use the partial Jensen-Shannon Divergence (JSD)—specifically, one-half of the full JSD equation. We have chosen to use partial JSD because we have chosen to focus our analysis on how the distribution of words in one dataset differs from another without averaging both sides of the comparison. Comparing distributions is useful for seeing how much one dataset deviates from a reference point (for example, the decade 1850 against the previous decade, 1840, and vice-versa) rather than measuring a balanced difference. It simplifies the calculation while still highlighting key differences, making it easier to interpret. JSD is especially helpful when comparing

texts from different time periods without averaging changes in both directions or masking asymmetrical shifts in word usage.

We hope that readers will become comfortable with the idea that algorithmic approaches to measuring key characteristics of a corpus are varied, and that the analyst can learn from comparing the results of working with different measurements.

Measuring Distance Between Words in a Corpus using JSD

First we will load our data and prepare it for our partial JSD function. The following code tokenizes and cleans both decades. It might take awhile to run.

```
# If we quit R studio, we will need to reload the libraries
library(tidyverse)
library(hansardr)
library(tidytext)
library(dhmeasures)
library(scales)

# Load Hansard data for the 1840s and 1850s
data("hansard_1840")
data("hansard_1850")

# Preprocess the 1840s Hansard data
# - unnest_tokens() tokenizes the text into one word per row
# - anti_join() removes common stop words
# - filter() removes words containing digits
# - mutate() labels all rows with the decade
hansard_1840 <- hansard_1840 %>%
  unnest_tokens(word, text) %>% # Break the text into words
  anti_join(stop_words) %>% # Remove stop words
  filter(!str_detect(word, "[:digit:]")) %>% # Remove words containing digits
  mutate(decade = 1840) # Label all rows as 1840

# Preprocess the 1850s Hansard data
# - unnest_tokens() tokenizes the text into one word per row
# - anti_join() removes common stop words
# - filter() removes words containing digits
# - mutate() labels all rows with the decade
hansard_1850 <- hansard_1850 %>%
  unnest_tokens(word, text) %>% # Break the text into words
  anti_join(stop_words) %>% # Remove stop words
  filter(!str_detect(word, "[:digit:]")) %>% # Remove words containing digits
  mutate(decade = 1850) # Label all rows as 1850

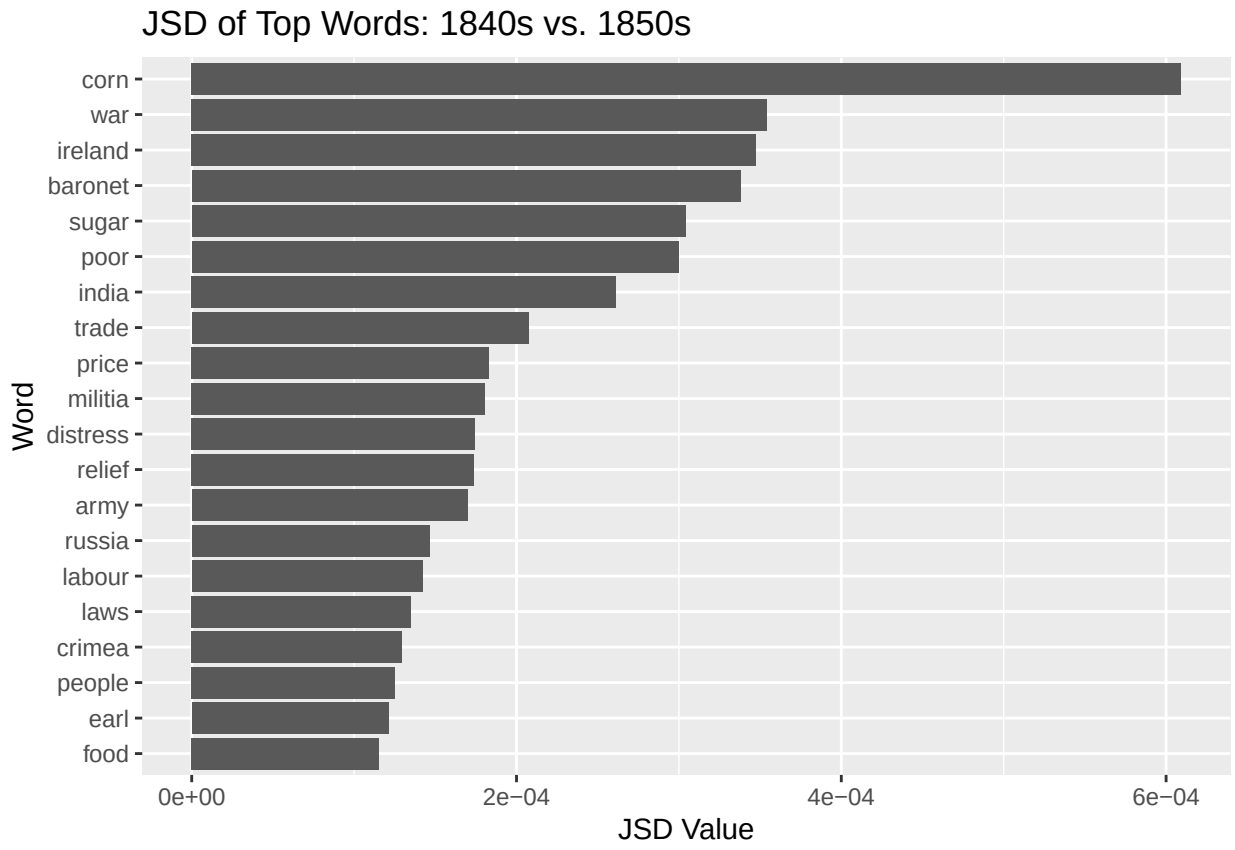
# combine both decades
hansard_combined <- bind_rows(hansard_1840, hansard_1850)
```

```r
# count each word within each decade
hansard_counts <- hansard_combined %>%
  count(decade, word)

# Calculate the JSD contribution of each word
# - jsd() calculates the Jensen-Shannon distance
# - rename() renames the JSD score column
output <- jsd(hansard_counts,
              group = "decade", # Compare the decades
              group_list = c(1840, 1850)) %>% # Compare the 1840s and 1850s
  rename(jsd = 2) # Rename the JSD score column

# Keep the words with the highest JSD scores
# - slice_max() keeps the top 20 words
word_dist_top <- output %>%
  slice_max(order_by = jsd, n = 20) # Keep the top 20 words

# Visualize the most distinctive words
# - reorder() orders the words by JSD score
# - geom_col() creates a bar chart
# - coord_flip() flips the axes for readability
# - labs() adds the title and axis labels
ggplot(word_dist_top,
       aes(x = reorder(word, jsd), # Order words by JSD score
           y = jsd)) + # Plot JSD scores on the y-axis
  geom_col() + # Create a bar chart
  coord_flip() + # Flip the axes
  labs(title = "JSD of Top Words: 1840s vs. 1850s", # Add a chart title
       x = "Word", # Label the x-axis
       y = "JSD Value") # Label the y-axis
```



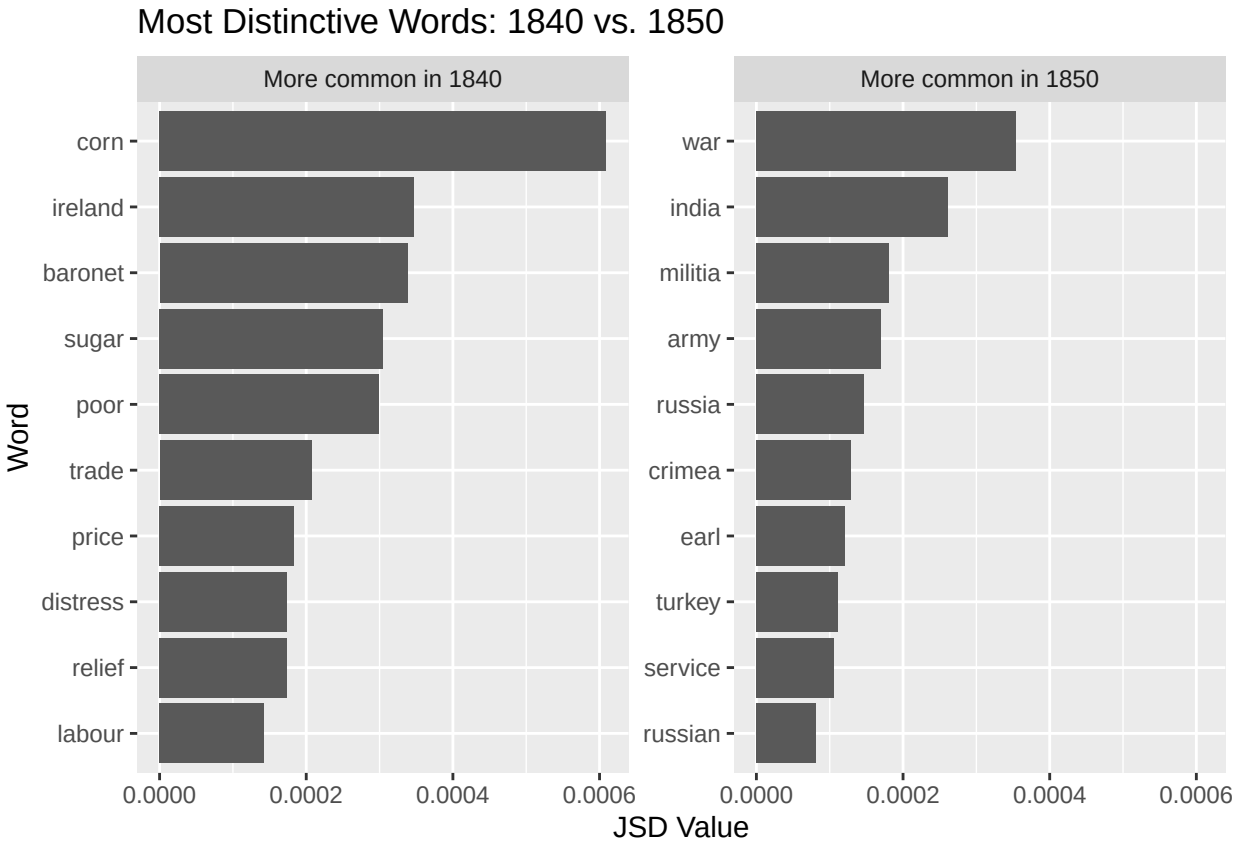

```
# Calculate the JSD contribution of each word
# - jsd() calculates the Jensen-Shannon distance
# - rename() renames the JSD score column
word_dist <- jsd(hansard_counts,
                 group = "decade", # Compare the decades
                 group_list = c(1840, 1850)) %>% # Compare the 1840s and 1850s
  rename(jsd = 2) # Rename the JSD score column

# Calculate the relative frequency of each word within each decade
# - group_by() groups the words by decade
# - mutate() converts word counts to proportions
# - select() keeps only the relevant columns
# - pivot_wider() creates one column for each decade
word_props <- hansard_counts %>%
  group_by(decade) %>% # Group by decade
  mutate(prop = n / sum(n)) %>% # Convert counts to proportions
  select(decade, word, prop) %>% # Keep the relevant columns
  pivot_wider(names_from = decade, # Create one column per decade
              values_from = prop, # Fill the columns with proportions
              values_fill = 0) # Replace missing values with zeros
```

```r
# Determine which decade each word is more common in
# - left_join() adds the relative frequencies
# - mutate() labels the decade in which each word is more common
word_dist <- word_dist %>%
  left_join(word_props, by = "word") %>% # Add the relative frequencies
  mutate(decade_more_common = if_else(`1840` > `1850`, # Compare the relative frequencies
                                      "More common in 1840",
                                      "More common in 1850")) # Label the more common decad

# Keep the most distinctive words from each decade
# - group_by() groups the words by decade
# - slice_max() keeps the top 10 words in each group
# - ungroup() removes the grouping
word_dist_top <- word_dist %>%
  group_by(decade_more_common) %>% # Group by the more common decade
  slice_max(jsd, n = 10) %>% # Keep the top 10 words
  ungroup() # Remove the grouping

# Visualize the most distinctive words
# - reorder() orders the words by JSD score
# - geom_col() creates a bar chart
# - coord_flip() flips the axes for readability
# - facet_wrap() creates one panel for each decade
# - scale_y_continuous() formats the y-axis labels
# - labs() adds the title and axis labels
ggplot(word_dist_top,
       aes(x = reorder(word, jsd), # Order words by JSD score
           y = jsd)) + # Plot JSD scores on the y-axis
  geom_col() + # Create a bar chart
  coord_flip() + # Flip the axes
  facet_wrap(~decade_more_common, # Create one panel per decade
             scales = "free_y") +
  scale_y_continuous(labels = label_number()) + # Format the y-axis labels
  labs(title = "Most Distinctive Words: 1840 vs. 1850", # Add a chart title
       x = "Word", # Label the x-axis
       y = "JSD Value") # Label the y-axis
```



Analysis - Jo?

The causes and implications of language changes are interesting to contemplate, and, as with much work in text mining, they require deeper reading of context in order to dive deeper. The point is that distance measurement lends us a metric to make concrete phenomena that are otherwise invisible – the pulse of rising and falling trends over time.

Exercises:

1) Take a single debate, such as the 1833 Debate on the Abolition of Slavery, and find the individual speakers. Use JSD to create comparisons between speakers. Remember to go back into the debates themselves and closely read the text before making definitive claims.

2) Up to this point we have analyzed one-word and two-word utterances (e.g. tokens and bigrams). Are there limitations to this approach when it comes to making meaning from historical data?

3) Use JSD to compare two decades of parliamentary debate. Identify the twenty words that contribute most to the divergence between the decades. What historical events or social changes might explain these differences?

4) Apply both tf-idf and JSD to the same pair of decades. Which words appear in both analyses? Which words appear in only one? What does this reveal about the assumptions of each method?

5) JSD identifies words that are statistically distinctive, but not necessarily important for a historical argument. Examine the top ten JSD words and suggest which three are most historically significant. Justify your reasoning using evidence from the debates.

6) What might go wrong with our analysis if we apply distance or divergence on two, separate decades like 1830 against 1850, or 1840 against 1860?

7) Think of a historically important event from the decade that does not appear among the highest-scoring tf-idf words. Why might tf-idf fail to highlight something that historians consider important?